# Spot the Difference

HIT137 Software Now

Assignment 03 Developer Documentation

*Tkinter · OpenCV · customtkinter · pytest*


**CHARLES DARWIN UNIVERSITY**

S126 HIT137 SOFTWARE NOW

SUBMITTED BY

Amsh Shrestha - S394695

Shanti Moktan - S395240

Surendra Sunar – S400227

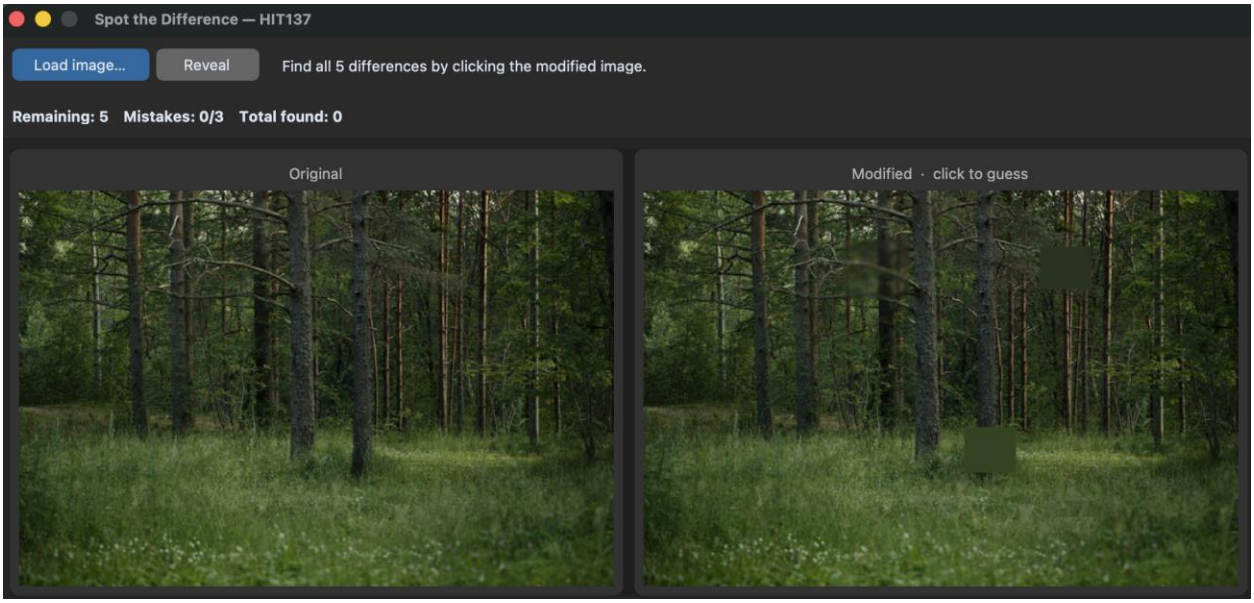
Nikesh Shrestha – S394589

SUBMITTED TO

Reim Sherif

# Table of Contents

# 1. Project Overview




Spot the Difference is a desktop game built with Python. The player loads any image; the application automatically generates a modified copy with five hidden differences applied using OpenCV image-processing operations. The player then clicks the modified image to locate all differences before exhausting three mistake allowances.

| Property | Value |
|---|---|
| Language | Python 3.12 |
| UI Framework | customtkinter 5.2.2 (wraps Tkinter) |
| Image Processing | OpenCV 4.13 + NumPy 2.4 |
| Test Framework | pytest 9.0.3 |
| Entry Point | python __main__.py |
| Differences per game | 5 (configurable) |
| Mistake allowance | 3 |

# 2. Project Structure

```
spot_the_difference/
├── __main__.py          # Entry point — calls src.run_app()
├── requirements.txt     # Pinned dependencies
├── conftest.py          # pytest configuration (empty)
├── src/
│   ├── __init__.py      # Exports run_app
│   ├── app.py           # run_app() — creates CTk root and UI
│   ├── models.py        # Rect, DifferenceRegion dataclasses
```

```
│   ├── generator.py     # DifferenceGenerator — places alterations
│   ├── alterations.py   # Alteration protocol + 6 concrete classes
│   └── ui.py            # SpotTheDifferenceUI — full Tkinter UI
└── tests/
    ├── test_models.py       # Unit tests for Rect & DifferenceRegion
    ├── test_alterations.py  # Unit tests for each alteration class
    └── test_generator.py    # Unit tests for DifferenceGenerator
```

# 3. Setup & Running

## 3.1 Installation

```
# 1. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt
```

## 3.2 Running the Application

```
python __main__.py
```

## 3.3 Running Tests

```
pytest                  # run all tests
pytest -v               # verbose output
pytest tests/test_alterations.py   # single file
```

# 4. Module Reference

## 4.1 src/models.py

Contains the core data structures used throughout the application.

**Rect**

A plain dataclass representing an axis-aligned rectangle.

| Member | Type | Description |
|---|---|---|
| x, y | int | Top-left corner coordinates |
| w, h | int | Width and height in pixels |
| x2 (property) | int | Right edge: x + w |
| y2 (property) | int | Bottom edge: y + h |
| clamp_within(width, height) | Rect | Returns a new Rect clipped to image bounds; guarantees w >= 1 and h >= 1 |

### DifferenceRegion

It pairs a Rect with metadata about a single applied alteration.

| Member | Type | Description |
|---|---|---|
| rect | Rect | Bounding box of the difference |
| alteration_name | str | Name of the alteration that was applied (e.g. "blur") |
| found | bool | Set to True when the player correctly clicks this region; defaults to False |
| cx, cy (props) | int | Centre-point of the rect |
| radius (prop) | int | Hit-test radius: max(12, max(w,h) * 0.55) |

## 4.2 src/alterations.py

It defines the **Alteration** structural protocol, the **BaseAlteration** base class, and six concrete alteration types. This module demonstrates **inheritance and polymorphism**: every alteration shares the same `apply()` interface; callers never need to know which subclass they hold.

### Alteration Protocol

```python
@runtime_checkable
class Alteration(Protocol):
    """Structural protocol — anything with a name and apply() qualifies."""
    name: str

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        """Apply an in-place alteration to `image_bgr` within `rect`.
        Must use OpenCV operations.
        """
```

A runtime_checkable Protocol requiring:

- **`name`** — str attribute identifying the alteration
- **`apply(image_bgr, rect)`** — modifies the given image in-place within rect using OpenCV

**BaseAlteration**

```python
class BaseAlteration:
    """
    Base class for all concrete alterations.

    Subclasses must override `name` and `apply`. Calling apply() on the
    base class directly raises NotImplementedError — demonstrating
    inheritance and polymorphism: the same apply() call dispatches to
    whichever subclass is actually in use at runtime.
    """
    name: str = "base"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        raise NotImplementedError(f"{self.__class__.__name__} must implement apply()")

    def _get_roi(self, image_bgr: np.ndarray, rect: Rect) -> np.ndarray:
        """Shared helper — extract the region of interest."""
        return image_bgr[rect.y : rect.y2, rect.x : rect.x2]
```

Concrete base class. Raises NotImplementedError on apply () — subclasses must override it. Provides the shared helper _get_roi(image_bgr, rect) which slices a NumPy view of the region of interest.

### Concrete Alteration Classes

```python
class BlurAlteration(BaseAlteration):
    name = "blur"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        k = max(3, (min(rect.w, rect.h) // 6) | 1)  # must be odd
        image_bgr[rect.y : rect.y2, rect.x : rect.x2] = cv2.GaussianBlur(roi, (k, k), 0)


class NoiseAlteration(BaseAlteration):
    name = "noise"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        noise = np.random.normal(loc=0.0, scale=80.0, size=roi.shape).astype(np.float32)
        noisy = np.clip(roi.astype(np.float32) + noise, 0, 255).astype(np.uint8)
        image_bgr[rect.y : rect.y2, rect.x : rect.x2] = noisy


class ColourShiftAlteration(BaseAlteration):
    name = "colour_shift"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV).astype(np.int16)
        hue_shift = np.random.randint(20, 45)
        hsv[:, :, 0] = (hsv[:, :, 0] + hue_shift) % 180
        hsv[:, :, 1] = np.clip(hsv[:, :, 1] + 50, 0, 255)
        image_bgr[rect.y : rect.y2, rect.x : rect.x2] = cv2.cvtColor(
            hsv.astype(np.uint8), cv2.COLOR_HSV2BGR
        )
```

```python
class EraseAlteration(BaseAlteration):
    name = "erase"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        border_pixels = np.concatenate(
            [roi[0, :], roi[-1, :], roi[:, 0], roi[:, -1]], axis=0
        )
        border_median = np.median(border_pixels, axis=0).astype(np.uint8)
        image_bgr[rect.y : rect.y2, rect.x : rect.x2] = border_median


class RotateAlteration(BaseAlteration):
    name = "rotate"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        image_bgr[rect.y : rect.y2, rect.x : rect.x2] = cv2.rotate(roi, cv2.ROTATE_180)


class SwapHalvesAlteration(BaseAlteration):
    name = "swap_halves"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        h = roi.shape[0]
        if h < 2:
            return
        cy = h // 2
        top = roi[:cy, :].copy()
        bottom = roi[cy:, :].copy()
        image_bgr[rect.y : rect.y + bottom.shape[0], rect.x : rect.x2] = bottom
        image_bgr[rect.y + bottom.shape[0] : rect.y2, rect.x : rect.x2] = top
```

| Class | name | Effect |
|---|---|---|
| BlurAlteration | blur | Applies Gaussian blur; kernel size auto-scales with rect size (must be odd) |
| NoiseAlteration | noise | Adds Gaussian noise (scale=80) to each pixel, clipped to [0,255] |
| ColourShiftAlteration | colour_shift | Converts to HSV, shifts hue 20–45 degrees and boosts saturation by 50 |
| EraseAlteration | erase | Fills the region with the median colour of its border pixels (camouflage effect) |

| | | |
|---|---|---|
| RotateAlteration | rotate | Rotates the region 180 degrees in-place using cv2.ROTATE_180 |
| SwapHalvesAlteration | swap_halves | Splits the region horizontally at the midpoint and swaps top and bottom halves |

```
DEFAULT_ALTERATIONS: list[Alteration] = [
    BlurAlteration(),
    NoiseAlteration(),
    EraseAlteration(),
    RotateAlteration(),
    SwapHalvesAlteration(),
    ColourShiftAlteration(),
]
```

**`DEFAULT_ALTERATIONS`** is a module-level list containing one instance of each of the six classes above. It is used by `DifferenceGenerator` when no custom list is supplied.

## 4.3 src/generator.py

Generator.py is responsible for placing non-overlapping alteration regions on a copy of the original image.

**_rects_overlap(a, b, pad=10)**

```
def _rects_overlap(a: Rect, b: Rect, pad: int = 10) -> bool:
    ax1, ay1, ax2, ay2 = a.x - pad, a.y - pad, a.x2 + pad, a.y2 + pad
    bx1, by1, bx2, by2 = b.x - pad, b.y - pad, b.x2 + pad, b.y2 + pad
    return not (ax2 <= bx1 or bx2 <= ax1 or ay2 <= by1 or by2 <= ay1)
```

It is a module-level helper which returns True if two Rects overlap when each is expanded by pad pixels on all sides. It is used to ensure differences are visually separated.

**DifferenceGenerator**

```python
class DifferenceGenerator:
    def __init__(
        self,
        *,
        num_differences: int = 5,
        alterations: list[Alteration] | None = None,
        rng: random.Random | None = None,
    ) -> None:
        self.num_differences = num_differences
        self.alterations = alterations or list(DEFAULT_ALTERATIONS)
        self.rng = rng or random.Random()

    def generate(self, original_bgr: np.ndarray) -> tuple[np.ndarray, list[DifferenceRegion]]:
        if original_bgr is None or original_bgr.size == 0:
            raise ValueError("Empty image")

        h, w = original_bgr.shape[:2]
        modified = original_bgr.copy()
        regions: list[DifferenceRegion] = []

        min_side = max(22, int(min(w, h) * 0.08))
        max_side = max(min_side + 1, int(min(w, h) * 0.18))

        max_attempts = 2000
        attempts = 0

        while len(regions) < self.num_differences and attempts < max_attempts:
            attempts += 1
            rw = self.rng.randint(min_side, max_side)
            rh = self.rng.randint(min_side, max_side)
            x = self.rng.randint(0, max(0, w - rw))
            y = self.rng.randint(0, max(0, h - rh))
            rect = Rect(x=x, y=y, w=rw, h=rh).clamp_within(w, h)

            if any(_rects_overlap(rect, r.rect) for r in regions):
                continue

            alteration = self.rng.choice(self.alterations)
            alteration.apply(modified, rect)
            regions.append(DifferenceRegion(rect=rect, alteration_name=alteration.name))

        if len(regions) != self.num_differences:
            raise RuntimeError(
                "Could not place non-overlapping differences; try a larger image."
            )

        return modified, regions
```

| Parameter | Default | Description |
|---|---|---|
| num_differences | 5 | How many differences to place per game |
| alterations | DEFAULT_ALTERATIONS | List of Alteration instances to pick from at random |
| rng | random.Random() | Random number generator — inject a seeded instance for reproducible tests |

**generate(original_bgr) → (modified, regions)**

Core method. Returns a tuple of:

- **modified** — a NumPy BGR array (deep copy of original) with alterations applied
- **regions** — a list of DifferenceRegion objects, one per placed difference

Placement algorithm:

- Validate the image is non-empty.
- Compute region size bounds as 8%–18% of the shorter image dimension.
- Attempt up to 2000 times to place a random rect that does not overlap any already-placed rect.
- Pick a random Alteration, apply it to the modified copy, and record a DifferenceRegion.
- Raise RuntimeError if num_differences regions cannot be placed within the attempt budget.

**Important**
The original image array is never modified. generate() operates on a deep copy (original_bgr.copy()) so callers can safely compare original vs modified.

### 4.4 src/app.py

```python
from __future__ import annotations

import customtkinter as ctk

from .ui import SpotTheDifferenceUI

def run_app() -> None:
    root = ctk.CTk()
    SpotTheDifferenceUI(root)
    root.mainloop()
```

app.py defines run_app(), the sole public entry point. It instantiates a customtkinter CTk root window, creates a SpotTheDifferenceUI, and starts the Tkinter event loop.

### src/__init__.py

```python
from .app import run_app

__all__ = ["run_app"]
```

__init__.py re-exports run_app so callers can write: from src import run_app

## 4.5 src/ui.py

```python
@dataclass
class DisplayImage:
    photo: ImageTk.PhotoImage
    scale: float
    width: int
    height: int

    def to_original(self, x_display: int, y_display: int) -> tuple[int, int]:
        return int(x_display / self.scale), int(y_display / self.scale)


def _bgr_to_photo(image_bgr: np.ndarray, max_side: int = 520) -> DisplayImage:
    h, w = image_bgr.shape[:2]
    scale = min(1.0, max_side / float(max(w, h)))
    dw, dh = int(w * scale), int(h * scale)
    rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    pil = Image.fromarray(rgb)
    if scale != 1.0:
        pil = pil.resize((dw, dh), Image.Resampling.LANCZOS)
    return DisplayImage(photo=ImageTk.PhotoImage(pil), scale=scale, width=dw, height=dh)


class SpotTheDifferenceUI:
    def __init__(self, root: ctk.CTk) -> None:
        self.root = root
        self.root.title("Spot the Difference — HIT137")
        self.root.resizable(False, False)

        self.generator = DifferenceGenerator()

        self.original_bgr: np.ndarray | None = None
        self.modified_bgr: np.ndarray | None = None
        self.regions: list[DifferenceRegion] = []

        self.total_found = 0
        self.mistakes = 0
        self.max_mistakes = 3
        self.guessing_enabled = False
```

```
    self._orig_display: DisplayImage | None = None
    self._mod_display: DisplayImage | None = None

    self._build()

def _build(self) -> None:
    # Top bar
    top = ctk.CTkFrame(self.root, corner_radius=0)
    top.pack(fill="x")

    ctk.CTkButton(
        top, text="Load image…", width=120, command=self.load_image
    ).pack(side="left", padx=(12, 6), pady=10)

    ctk.CTkButton(
        top, text="Reveal", width=90,
        fg_color="gray40", hover_color="gray55",
        command=self.reveal
    ).pack(side="left", padx=(0, 12), pady=10)

    self.status_var = tk.StringVar(value="Load an image to start.")
    ctk.CTkLabel(top, textvariable=self.status_var, anchor="w").pack(
        side="left", padx=(8, 0)
    )

    # Score bar
    score_frame = ctk.CTkFrame(self.root, corner_radius=0, fg_color="gray17")
    score_frame.pack(fill="x")

    self.score_var = tk.StringVar(value="Remaining: 0    Mistakes: 0/3    Total found: 0")
    ctk.CTkLabel(
        score_frame, textvariable=self.score_var,
        font=ctk.CTkFont(size=13, weight="bold")
    ).pack(pady=6, padx=12, anchor="w")
```

```
    # Main body frame — now stored as self.body to avoid AttributeError
    self.body = ctk.CTkFrame(self.root, corner_radius=0, fg_color="gray12")
    self.body.pack(fill="both", expand=True, padx=10, pady=10)

    left = ctk.CTkFrame(self.body, corner_radius=8, fg_color="gray20")
    left.pack(side="left", fill="both", expand=True, padx=(0, 5))

    right = ctk.CTkFrame(self.body, corner_radius=8, fg_color="gray20")
    right.pack(side="left", fill="both", expand=True, padx=(5, 0))

    ctk.CTkLabel(left, text="Original", text_color="gray70").pack(pady=(6, 2))
    ctk.CTkLabel(right, text="Modified  ·  click to guess", text_color="gray70").pack(pady=(6, 2))

    self.canvas_orig = tk.Canvas(left, bg="#1a1a1a", highlightthickness=0)
    self.canvas_orig.pack(padx=8, pady=(0, 8))

    self.canvas_mod = tk.Canvas(right, bg="#1a1a1a", highlightthickness=0)
    self.canvas_mod.pack(padx=8, pady=(0, 8))

    self.canvas_mod.bind("<Button-1>", self.on_click_modified)

def _update_score_labels(self) -> None:
    remaining = sum(1 for r in self.regions if not r.found)
    self.score_var.set(
        f"Remaining: {remaining}    Mistakes: {self.mistakes}/{self.max_mistakes}    Total found: {self.total_foun
    )

def _clear_canvases(self) -> None:
    self.canvas_orig.delete("all")
    self.canvas_mod.delete("all")

def _render_images(self) -> None:
    if self.original_bgr is None or self.modified_bgr is None:
        return
    self._orig_display = _bgr_to_photo(self.original_bgr)
    self._mod_display = _bgr_to_photo(self.modified_bgr)
```

```
    self._clear_canvases()
    for canvas, display in (
        (self.canvas_orig, self._orig_display),
        (self.canvas_mod, self._mod_display),
    ):
        canvas.config(width=display.width, height=display.height)
        canvas.create_image(0, 0, image=display.photo, anchor="nw", tags=("img",))

def load_image(self) -> None:
    path = filedialog.askopenfilename(
        title="Choose an image",
        filetypes=[("Images", "*.jpg *.jpeg *.png *.bmp"), ("All files", "*.*")],
    )
    if not path:
        return

    img = cv2.imread(str(Path(path)), cv2.IMREAD_COLOR)
    if img is None:
        messagebox.showerror("Load failed", "Could not read this image.")
        return

    try:
        modified, regions = self.generator.generate(img)
    except Exception as e:
        messagebox.showerror("Generation failed", str(e))
        return

    self.original_bgr = img
    self.modified_bgr = modified
    self.regions = regions
    self.mistakes = 0
    self.total_found = 0          # reset total found
    self.guessing_enabled = True  # re-enable guessing

    self.status_var.set("Find all 5 differences by clicking the modified image.")
    self._render_images()
    self._update_score_labels()
```

```
def _draw_circle_on_both(self, region: DifferenceRegion, color: str) -> None:
    if self._orig_display is None or self._mod_display is None:
        return
    s = self._orig_display.scale
    cx = int(region.cx * s)
    cy = int(region.cy * s)
    r = int(region.radius * s)
    for canvas in (self.canvas_orig, self.canvas_mod):
        canvas.create_oval(cx - r, cy - r, cx + r, cy + r, outline=color, width=3)

def on_click_modified(self, event: tk.Event) -> None:
    if not self.guessing_enabled or self._mod_display is None:
        return

    ox, oy = self._mod_display.to_original(event.x, event.y)

    hit = next(
        (
            r for r in self.regions
            if not r.found
            and ((ox - r.cx) ** 2 + (oy - r.cy) ** 2) ** 0.5 <= r.radius
        ),
        None,
    )

    if hit is None:
        self.mistakes += 1
        self._update_score_labels()

        if self.mistakes >= self.max_mistakes:
            # Reveal remaining differences and show game over popup
            self._reveal_remaining(show_popup=True)
        else:
            self.status_var.set(f"Not quite — {self.max_mistakes - self.mistakes} guess(es) left.")
        return
```

```
        # Correct guess
        hit.found = True
        self.total_found += 1
        self._draw_circle_on_both(hit, color="#ff4444")

        remaining = sum(1 for r in self.regions if not r.found)
        if remaining == 0:
            self._end_game(won=True)   # keep your existing win method
        else:
            self.status_var.set(f"Found one! {remaining} to go.")
        self._update_score_labels()

    def reveal(self) -> None:
        """Reveal all unfound differences with blue circles and disable guessing."""
        self._reveal_remaining(show_popup=False)

    def _end_game(self, won: bool) -> None:
        """Disable guessing and show a final message."""
        self.guessing_enabled = False
        if won:
            messagebox.showinfo("🎉 You won!", f"You found all {len(self.regions)} differences!")
            self.status_var.set(f"You won! Found all {len(self.regions)}. Load a new image to play again.")
        else:
            messagebox.showwarning("💀 Game Over", f"Too many mistakes ({self.max_mistakes}). Better luck next time!")
            self.status_var.set(f"Game over! You made {self.max_mistakes} mistakes. Load a new image to try again.")

    def _reveal_remaining(self, show_popup: bool = False) -> None:
        """Draw blue circles on all unfound differences and disable guessing."""
        if not self.regions:
            return

        for r in self.regions:
            if not r.found:
                self._draw_circle_on_both(r, color="#4499ff")

        self.guessing_enabled = False

        if show_popup:
            remaining = sum(1 for r in self.regions if not r.found)
            messagebox.showinfo(
                "Game Over",
                f"You've used all {self.max_mistakes} mistakes.\n"
                f"{remaining} difference(s) remaining have been revealed."
            )
            self.status_var.set(f"Game over — revealed {remaining} remaining differences. Load a new image.")
        else:
            self.status_var.set("Revealed. Load a new image to restart.")
```

It contains all user-interface logic. The module is structured around two helpers and one main class.

### DisplayImage (dataclass)

It holds a rendered PhotoImage together with the scale factor and pixel dimensions used to fit it on-screen. Its to_original(x, y) method converts display-space click coordinates back to original-image coordinates.

### _bgr_to_photo(image_bgr, max_side=520)

It converts an OpenCV BGR array to a Tkinter-compatible PhotoImage. Scales the image down (never up) so its longest side does not exceed max_side pixels. Returns a DisplayImage.

### SpotTheDifferenceUI

It is the main application class. It is constructed with a CTk root window.

| Method | Description |
|---|---|
| _build() | Constructs all widgets: top bar (Load / Reveal buttons, status label), score bar, and dual-canvas body |
| load_image() | Opens a file dialog, reads the chosen image with cv2.imread, calls generator.generate(), resets game state, and renders both canvases |
| _render_images() | Converts both BGR arrays to PhotoImages and draws them on the two canvases |
| on_click_modified(event) | Hit-tests the click against unfound regions (Euclidean distance <= radius). Correct: marks found, draws red circle, checks win. Wrong: increments mistakes, triggers game over if limit reached |
| _draw_circle_on_both(region, color) | Draws an oval on both canvases at the region's scaled centre and radius |
| reveal() | Public button handler — calls _reveal_remaining(show_popup=False) |
| _reveal_remaining(show_popup) | Draws blue circles on all unfound regions and disables guessing. Optionally shows a game-over popup |
| _end_game(won) | Disables guessing and shows a win or loss message box |
| _update_score_labels() | Updates the score bar with current remaining count, mistakes, and total found |

### Game State Fields

| Field | Type | Purpose |
|---|---|---|
| original_bgr | ndarray \| None | Unmodified image loaded from disk |
| modified_bgr | ndarray \| None | Copy with alterations applied |
| regions | list[DifferenceRegion] | Ground-truth locations of all differences |
| mistakes | int | Count of wrong clicks; resets on load_image |
| total_found | int | Count of correct guesses in this game |
| max_mistakes | int (3) | Game-over threshold |

| | | |
|---|---|---|
| guessing_enabled | bool | False before image is loaded and after game ends; guards on_click_modified |

## 5. Test Suite

All tests live under tests/ and require no external fixtures or network access. They are discovered automatically by pytest.

### 5.1 test_models.py — TestRect & TestDifferenceRegion

```
class TestRect:
    def test_x2_y2(self):
        r = Rect(x=10, y=20, w=30, h=40)
        assert r.x2 == 40
        assert r.y2 == 60

    def test_clamp_fits_inside(self):
        r = Rect(x=10, y=10, w=50, h=50).clamp_within(200, 200)
        assert r.x == 10 and r.y == 10 and r.w == 50 and r.h == 50

    def test_clamp_origin_negative(self):
        r = Rect(x=-5, y=-10, w=30, h=30).clamp_within(200, 200)
        assert r.x >= 0 and r.y >= 0

    def test_clamp_overflow_right(self):
        r = Rect(x=190, y=10, w=50, h=50).clamp_within(200, 200)
        assert r.x + r.w <= 200

    def test_clamp_overflow_bottom(self):
        r = Rect(x=10, y=190, w=50, h=50).clamp_within(200, 200)
        assert r.y + r.h <= 200

    def test_clamp_minimum_size(self):
        # Even a zero-size rect should come out with w>=1, h>=1
        r = Rect(x=0, y=0, w=0, h=0).clamp_within(200, 200)
        assert r.w >= 1 and r.h >= 1
```

```python
class TestDifferenceRegion:
    def test_cx_cy(self):
        region = DifferenceRegion(rect=Rect(x=10, y=20, w=40, h=60), alteration_name="blur")
        assert region.cx == 30  # 10 + 40//2
        assert region.cy == 50  # 20 + 60//2

    def test_radius_minimum(self):
        # Small rect should still have radius >= 12
        region = DifferenceRegion(rect=Rect(x=0, y=0, w=5, h=5), alteration_name="blur")
        assert region.radius >= 12

    def test_radius_scales_with_rect(self):
        small = DifferenceRegion(rect=Rect(x=0, y=0, w=20, h=20), alteration_name="blur")
        large = DifferenceRegion(rect=Rect(x=0, y=0, w=100, h=100), alteration_name="blur")
        assert large.radius > small.radius

    def test_found_defaults_false(self):
        region = DifferenceRegion(rect=Rect(x=0, y=0, w=20, h=20), alteration_name="blur")
        assert region.found is False
```

| Test | Verifies |
|---|---|
| test_x2_y2 | x2 = x + w, y2 = y + h |
| test_clamp_fits_inside | In-bounds rect unchanged by clamp_within |
| test_clamp_origin_negative | Negative origin clamped to 0 |
| test_clamp_overflow_right/bottom | Right/bottom edges clamped within image dimensions |
| test_clamp_minimum_size | Zero-size rect produces w >= 1, h >= 1 |
| test_cx_cy | Centre-point properties compute correctly |
| test_radius_minimum | Radius is at least 12 for small rects |
| test_radius_scales_with_rect | Larger rects have larger radii |
| test_found_defaults_false | found field starts as False |

## 5.2 test_alterations.py — TestBaseAlteration, TestEachAlteration, TestDefaultAlterations

```python
def _blank_image(h: int = 100, w: int = 100) -> np.ndarray:
    """Solid mid-grey BGR image."""
    return np.full((h, w, 3), 128, dtype=np.uint8)


def _rect(x: int = 10, y: int = 10, w: int = 40, h: int = 40) -> Rect:
    return Rect(x=x, y=y, w=w, h=h)


class TestBaseAlteration:
    def test_apply_raises(self):
        with pytest.raises(NotImplementedError):
            BaseAlteration().apply(_blank_image(), _rect())

    def test_get_roi_shape(self):
        img = _blank_image()
        r = _rect(10, 10, 30, 20)
        roi = BaseAlteration()._get_roi(img, r)
        assert roi.shape == (20, 30, 3)
```

```python
class TestEachAlteration:
    """Each alteration should visibly change the ROI pixels."""

    ALTERATIONS = [
        BlurAlteration(),
        NoiseAlteration(),
        ColourShiftAlteration(),
        EraseAlteration(),
        RotateAlteration(),
        SwapHalvesAlteration(),
    ]

    @pytest.mark.parametrize("alteration", ALTERATIONS, ids=lambda a: a.name)
    def test_modifies_image(self, alteration):
        img = _blank_image()
        # Add structure so rotate/swap actually have something to change
        img[10:50, 10:50] = np.random.randint(0, 255, (40, 40, 3), dtype=np.uint8)
        original_roi = img[10:50, 10:50].copy()

        alteration.apply(img, _rect())

        modified_roi = img[10:50, 10:50]
        assert not np.array_equal(original_roi, modified_roi), (
            f"{alteration.name} did not change the image"
        )

    @pytest.mark.parametrize("alteration", ALTERATIONS, ids=lambda a: a.name)
    def test_empty_roi_no_crash(self, alteration):
        """Zero-size rect should be a silent no-op, not a crash."""
        img = _blank_image()
        alteration.apply(img, Rect(x=0, y=0, w=0, h=0))
```

```
    @pytest.mark.parametrize("alteration", ALTERATIONS, ids=lambda a: a.name)
    def test_does_not_touch_outside_roi(self, alteration):
        """Pixels outside the rect must be untouched."""
        img = _blank_image()
        outside_before = img[0:10, 0:10].copy()
        alteration.apply(img, _rect(x=20, y=20, w=30, h=30))
        assert np.array_equal(img[0:10, 0:10], outside_before), (
            f"{alteration.name} wrote outside its rect"
        )


class TestDefaultAlterations:
    def test_has_six_entries(self):
        assert len(DEFAULT_ALTERATIONS) == 6

    def test_all_are_base_alteration(self):
        assert all(isinstance(a, BaseAlteration) for a in DEFAULT_ALTERATIONS)

    def test_names_are_unique(self):
        names = [a.name for a in DEFAULT_ALTERATIONS]
        assert len(names) == len(set(names))
```

| Test | Verifies |
|---|---|
| test_apply_raises | BaseAlteration.apply() raises NotImplementedError |
| test_get_roi_shape | _get_roi returns correct slice shape |
| test_modifies_image [×6] | Each alteration visibly changes the ROI (parametrized over all 6 classes) |
| test_empty_roi_no_crash [×6] | Zero-size rect is a silent no-op, not an exception |
| test_does_not_touch_outside_roi [×6] | Pixels outside the given rect are never modified |
| test_has_six_entries | DEFAULT_ALTERATIONS has exactly 6 entries |
| test_all_are_base_alteration | Every entry is an instance of BaseAlteration |
| test_names_are_unique | All alteration names are distinct |

## 5.3 test_generator.py — TestRectsOverlap & TestDifferenceGenerator

```python
def _image(h: int = 300, w: int = 300) -> np.ndarray:
    return np.random.randint(0, 255, (h, w, 3), dtype=np.uint8)


class TestRectsOverlap:
    def test_clearly_separate(self):
        a = Rect(x=0, y=0, w=20, h=20)
        b = Rect(x=100, y=100, w=20, h=20)
        assert not _rects_overlap(a, b, pad=0)

    def test_touching_edge(self):
        a = Rect(x=0, y=0, w=20, h=20)
        b = Rect(x=20, y=0, w=20, h=20)
        # With pad=0 they share an edge — counts as overlap
        assert _rects_overlap(a, b, pad=0)

    def test_overlapping(self):
        a = Rect(x=0, y=0, w=50, h=50)
        b = Rect(x=25, y=25, w=50, h=50)
        assert _rects_overlap(a, b)

    def test_padding_makes_nearby_overlap(self):
        a = Rect(x=0, y=0, w=20, h=20)
        b = Rect(x=25, y=0, w=20, h=20)
        # Gap of 5px — pad=10 should flag as overlap
        assert _rects_overlap(a, b, pad=10)
```

```python
class TestDifferenceGenerator:
    def test_returns_correct_number_of_regions(self):
        gen = DifferenceGenerator(rng=random.Random(42))
        _, regions = gen.generate(_image())
        assert len(regions) == 5

    def test_modified_differs_from_original(self):
        gen = DifferenceGenerator(rng=random.Random(42))
        img = _image()
        modified, _ = gen.generate(img)
        assert not np.array_equal(img, modified)

    def test_original_not_mutated(self):
        """generate() should never touch the original array."""
        gen = DifferenceGenerator(rng=random.Random(42))
        img = _image()
        original_copy = img.copy()
        gen.generate(img)
        assert np.array_equal(img, original_copy)

    def test_regions_do_not_overlap(self):
        gen = DifferenceGenerator(rng=random.Random(42))
        _, regions = gen.generate(_image())
        for i, a in enumerate(regions):
            for j, b in enumerate(regions):
                if i != j:
                    assert not _rects_overlap(a.rect, b.rect), (
                        f"Regions {i} and {j} overlap"
                    )
```

```python
    def test_all_regions_within_image(self):
        img = _image(300, 300)
        gen = DifferenceGenerator(rng=random.Random(42))
        _, regions = gen.generate(img)
        h, w = img.shape[:2]
        for r in regions:
            assert r.rect.x >= 0 and r.rect.y >= 0
            assert r.rect.x2 <= w and r.rect.y2 <= h

    def test_raises_on_empty_image(self):
        gen = DifferenceGenerator()
        with pytest.raises(ValueError, match="Empty image"):
            gen.generate(np.array([]))

    def test_raises_on_tiny_image(self):
        # 10x10 is impossible to fit 5 non-overlapping regions
        gen = DifferenceGenerator()
        with pytest.raises(RuntimeError, match="Could not place"):
            gen.generate(_image(10, 10))

    def test_different_seed_gives_different_layout(self):
        img = _image()
        _, regions_a = DifferenceGenerator(rng=random.Random(1)).generate(img.copy())
        _, regions_b = DifferenceGenerator(rng=random.Random(2)).generate(img.copy())
        positions_a = [(r.rect.x, r.rect.y) for r in regions_a]
        positions_b = [(r.rect.x, r.rect.y) for r in regions_b]
        assert positions_a != positions_b

    def test_all_regions_start_unfound(self):
        gen = DifferenceGenerator(rng=random.Random(42))
        _, regions = gen.generate(_image())
        assert all(not r.found for r in regions)

    def test_custom_num_differences(self):
        gen = DifferenceGenerator(num_differences=3, rng=random.Random(42))
        _, regions = gen.generate(_image())
        assert len(regions) == 3
```

| Test | Verifies |
|---|---|
| test_clearly_separate | Far-apart rects do not flag as overlapping |
| test_touching_edge | Shared edge counts as overlap with pad=0 |
| test_overlapping | Intersecting rects correctly flagged |
| test_padding_makes_nearby_overlap | A 5 px gap becomes overlap when pad=10 |
| test_returns_correct_number_of_regions | generate() returns exactly num_differences regions |

| | |
|---|---|
| test_modified_differs_from_original | Returned modified array is not pixel-equal to the input |
| test_original_not_mutated | The original array is unchanged after generate() |
| test_regions_do_not_overlap | No two placed regions overlap each other |
| test_all_regions_within_image | Every region rect stays within image bounds |
| test_raises_on_empty_image | ValueError raised for an empty numpy array |
| test_raises_on_tiny_image | RuntimeError raised when image is too small to fit 5 regions |
| test_different_seed_gives_different_layout | Different RNG seeds produce different region layouts |
| test_all_regions_start_unfound | All regions begin with found=False |
| test_custom_num_differences | num_differences=3 produces exactly 3 regions |

## 6. Dependencies

```
customtkinter==5.2.2
darkdetect==0.8.0
iniconfig==2.3.0
numpy==2.4.4
opencv-python==4.13.0.92
packaging==26.2
pillow==12.2.0
pluggy==1.6.0
Pygments==2.20.0
pytest==9.0.3
```

| Package | Version | Purpose |
|---|---|---|
| customtkinter | 5.2.2 | Modern-styled Tkinter widgets and theming |
| opencv-python | 4.13.0.92 | Image read/write and all pixel-level operations |
| numpy | 2.4.4 | Array manipulation for alteration arithmetic |
| pillow | 12.2.0 | Bridge between OpenCV arrays and Tkinter PhotoImage |
| darkdetect | 0.8.0 | OS dark-mode detection (required by customtkinter) |
| pytest | 9.0.3 | Test runner |
| iniconfig, packaging, pluggy, pygments | various | pytest transitive dependencies |

# 7. Extending the Project

## 7.1 Adding a New Alteration

Create a subclass of BaseAlteration in alterations.py:

```
class MyAlteration(BaseAlteration):
    name = "my_alteration"

    def apply(self, image_bgr: np.ndarray, rect: Rect) -> None:
        roi = self._get_roi(image_bgr, rect)
        if roi.size == 0:
            return
        # ... your OpenCV operation here ...
```

Then add an instance to DEFAULT_ALTERATIONS or pass a custom list to DifferenceGenerator.

## 7.2 Changing the Number of Differences

Pass num_differences to DifferenceGenerator in ui.py:

```
self.generator = DifferenceGenerator(num_differences=7)
```

## 7.3 Seeding for Reproducibility

Inject a seeded random.Random instance — useful in tests or demonstrations:

```
import random
gen = DifferenceGenerator(rng=random.Random(42))
modified, regions = gen.generate(image)
```

# 8. Game Flow Summary

| # | Step |
|---|---|
| 1 | User clicks “Load image…” → file dialog opens |
| 2 | cv2.imread loads the file as a BGR array |
| 3 | DifferenceGenerator.generate() creates a modified copy with 5 random alterations |
| 4 | Both images are scaled and displayed side-by-side; game state is reset |
| 5 | Player clicks the modified (right) canvas |
| 6a | Hit: DifferenceRegion.found = True; red circle drawn on both canvases; score updated |
| 6b | Miss: mistakes incremented; if mistakes >= 3, all remaining differences revealed and game ends |
| 7a | Win: all 5 found → win popup shown; guessing disabled |
| 7b | Reveal button: all unfound regions shown as blue circles; guessing disabled immediately |